National University of Computer and Emerging Sciences

FAST Islamabad School Of Computing | Spring 2026

CS-1004 - Object-Oriented Programming (CS Department)

Assignment 3

Submission Deadline : *11:59 PM - 17th April, 2026*

Instructions :

1. Assignments are to be done in groups of two. Fill out team members in the form uploaded in Google Classroom. Batch 25 students are not permitted to make cross-sectional groups, but Batch 24 & above are permitted. You must complete this assignment by yourself. The code you write must be your own and you must understand each part of your code.You are encouraged, however, to get help from the instructional staff through Google Classroom.
2. Do not use any external libraries such as string, cstring, vector, cmath, algorithms, etc. **unless** specified by a question, and only for that question only.
3. Plagiarism: Plagiarism of any kind (copying from others, copying from the internet or using AI, etc.) is not allowed. If found plagiarised, you will be awarded zero marks in the whole assignment category. Repeating such an act can lead to strict disciplinary actions and failure in the course. Do note that AI and plagiarism will be thoroughly checked across the batch and different LLMs. If any hint of similarity is found, strict action will be taken.
4. A demo for this assignment will be held, where plagiarism and solutions to the assignment's questions will be assessed, and marks for the assignment will be given accordingly.
5. Please start early; otherwise, you will struggle with the assignment.
6. Keep an eye out for the comments on the assignment. Sometimes queries are asked in the comments, which add constraints to the questions of the assignment.
7. The query deadline will be the 10th Of April - 5 PM. No query will be entertained after that.
8. All queries must be asked only in the Google Classroom comment section for the assignment, allowing others with similar questions to be able to have the answers to their questions.
9. If you have a query, kindly check if anyone has already asked a similar query beforehand.
10. Make **separate** .cpp and .h files for each question. Rest assured, submission instructions will be given a few days before the deadline.

Q1: The Primordial Grid (120 Marks)

Introduction: The Inspiration (Conway's Game of Life)

Before diving into the complexities of the Primordial Grid, it is essential to understand its ancestor: Conway's Game of Life. Devised by mathematician John Conway in 1970, it is a "zero-player game," meaning its evolution is determined by its initial state, requiring no further input.

In the standard version:

1. The World: A 2D grid of "cells" that are either Alive or Dead.
2. The Rules: In every "tick" of time, a cell looks at its 8 immediate neighbors:

- Underpopulation: Any live cell with fewer than two live neighbors dies.
- Survival: Any live cell with two or three live neighbors lives on to the next generation.
- Overpopulation: Any live cell with more than three live neighbors dies.
- Reproduction: Any dead cell with exactly three live neighbors becomes a live cell.

While Conway's version uses global rules and single-pixel cells, our assignment evolves this concept. Here, the "rules" are no longer global; they are encapsulated behaviors inside different classes. Each organism is a multi-tiled object that "thinks" for itself.

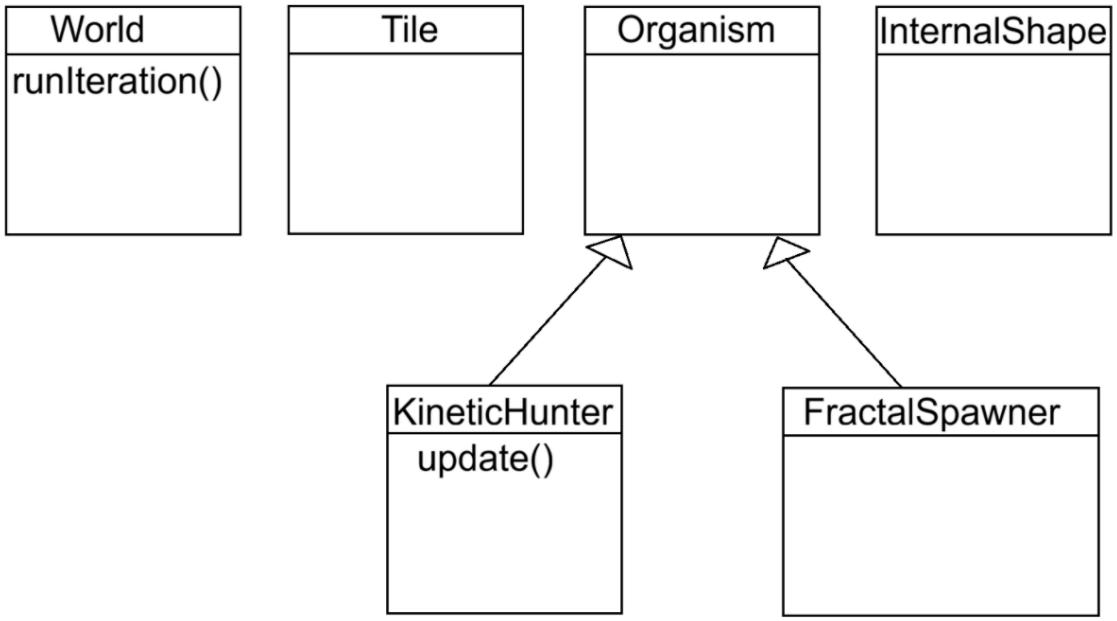
Your Mission

In a forgotten digital petri dish, multiple life forms compete for survival. Unlike the traditional Conway's Game of Life where a single cell's state is determined by a 3x3 neighborhood, these "Primordial Organisms" are complex entities with their own internal structures, varying sizes, and autonomous decision-making logic.

You are tasked with building a simulation where organisms of different species interact with a global environment (The World). Each organism must perceive its surroundings and decide whether to grow, shrink, move, or reproduce based on the resources available in the grid.

Part 1: The Diagram

You are provided with a partially-filled UML Class Diagram of the following scenario. After reading through the scenario, complete the diagram with appropriate attributes, methods, and relationships.



Part 2: The Environment

2.1 The Tile Struct

Define a `struct Tile` that represents a single point in the 2D world:

- `float nutrientLevel`: (0.0 to 100.0) Determines if life can be sustained.
- `float toxicity`: (0.0 to 100.0) High levels cause organisms to shrink or die.
- `Organism* occupant`: A pointer to the organism currently covering this tile. If multiple organisms overlap, this should point to the most "dominant" one.

2.2 The World Class

The `World` class manages the simulation:

- A 2D dynamic array of `Tile` objects (the grid).
- A dynamic array of `Organism*` (the population).
- `void runIteration()`: Iterates through all organisms and calls their update method, passing the entire Grid and its dimensions.

Part 3: The Organism Hierarchy

3.1 Base Class: Organism

Every organism must inherit from this base class. It contains:

- Attributes: `int x, y` (Top-left position), `int width, int height`.
- `bool** internalShape`: A 2D dynamic bitmask. If `internalShape[i][j]` is true, the organism occupies that relative coordinate in the World.
- Function: `virtual void update(Tile** world, int worldWidth, int worldHeight)`

- Helper Function: `float getAverageNutrients(Tile** world)` : Calculates the average nutrient level of all tiles currently covered by the organism's internalShape.

3.2 Derived Class: The Fractal Sprawler (The Producer)

- Behavior: It is stationary but expands.

Update Logic:

1. Scan a radius of 2 * width around its center.
2. If `averageNutrients > 70`, it Grows: Reallocate `internalShape` to increase width and height by 2.
3. If `averageNutrients < 30`, it Shrinks: Decrease dimensions. If size reaches 0x0, the object is deleted.
4. Reproduction: If dimensions reach 5x5, it creates a new `FractalSprawler` at a nearby empty location and resets its own size to 2x2.

3.3 Derived Class: The Kinetic Hunter (The Consumer)

- Behavior: It moves and "eats" other organisms.

Update Logic:

1. Scan a radius of 4 * width.
2. Movement: If it detects a `FractalSprawler`, it updates its x, y coordinates to move closer to it.
3. Interaction: If its `internalShape` overlaps with a `FractalSprawler`, it drains `nutrientLevel` from the world tiles and reduces the Sprawler's size.

Constraints & Requirements

1. Memory Management: You must manually manage all 2D dynamic arrays. Every time an organism "Grows" or "Shrinks," the old internalShape must be properly deallocated before the new size is allocated.
2. No Standard Libraries: Do NOT use `<vector>`, `<string>`, or `<algorithm>`. Do NOT use `std::vector`. Use raw pointers only.
3. The Update Loop: The World must not dictate how an organism behaves. The World provides the data, and the Organism's overridden update function contains the "intelligence."
4. Generality: The World should store all organisms in a single `Organism**` array. You must use virtual functions to ensure the correct update and draw logic is called.

Expected Output

The program should display the grid in the console for each iteration.

- Use Green Squares for Producer (Fractal Sprawler) tiles.
- Use Red Squares for Consumer (Kinetic Hunter) tiles.
- Use Black Squares for empty tiles.
- Display the current "Total Population" and "Average World Toxicity" after every iteration.

Q2: The Aethelgard Simulator (200 Marks)

1. The Long Winter Approaches

The long summer has ended, and the chill of the Abyssal Swarm creeps over the fractured realms of Aethelgard. Ten great houses, blinded by their thirst for the Obsidian Crown, bleed each other in petty wars of succession and spite, oblivious to the ancient nightmare gathering in the shadowed north. As the newly appointed Grand Scholar of the Citadel, it falls upon you to divine the fate of the realm. You must construct a mechanism to foresee the continent's bitter politics, grueling wars, and the inevitable, catastrophic clash with the Swarm. But tread carefully—the tools at your disposal are raw, unforgiving, and deeply volatile.

Your task is to build the Aethelgard Simulator, a complex, deterministic, console-based C++ engine that manages these squabbling factions (Realms) and their diverse subjects (Lords, Soldiers, Sages, Assassins). You will breathe life into a volatile political map as it spirals toward the ultimate endgame invasion.

The core challenge of this assignment is memory and architecture. You will build this simulation under strict constraints to test your deep understanding of raw dynamic memory management, data-only inheritance, and object composition without relying on standard safety features.

2. The Architecture of Fate

The history of Aethelgard is not written by the chaotic whims of chance, but forged by the inevitable gears of destiny. Every life drawn into the conflict, every blade swung in the mud, and every whispered plot in the dark corridors of the keeps unfolds exactly as pre-ordained. As the Grand Scholar, you do not guess the morrow; you calculate its certainty.

The Aethelgard engine class serves as the central manager of the simulation. The execution must be perfectly reproducible. Given the same starting configuration, the simulation must generate identical output every time.

2.1 The Bloodlines and Steel (Entity & Asset Inheritance)

Within the realm, a stark line divides the living from the lifeless metal they yield. Every soul that draws breath and every siege weapon dragged to the gates is born with an identity—a unique mark in the ledgers of time. But blood dictates destiny. The animate and the inanimate share nothing but a name, branching immediately into separate existences.

All classes in the simulation fundamentally descend from a base `Entity` class, inheriting only an `id` and a `name`. From this base class, the architecture splits into two primary branches:

1. The Person Branch: Inherits from `Entity` and adds attributes like `age` and `isAlive`.
2. The Asset Branch: Inherits from `Entity` and adds attributes like `hp`, `maxHp`, `armor`, and `maintenanceCost`.

2.2 The Hierarchy of Power (Derived Characters)

Aethelgard is a continent built on cruel hierarchies. The lowly foot soldier marching to the slaughter knows nothing of the grand strategies plotted by the landed lords in high towers, just as the silent assassin striking from the shadows cares little for the battlefield wisdom of the sages. Yet occasionally, legends are born—champions who master both the noble courts and the bloody vanguard, or terrifying juggernauts of war that dominate the seas as floating fortresses.

The components must be perfectly modeled by explicit data structures descending from the base entity classes. You must define this architecture precisely:

The Armies (`MilitaryUnit`) Every military unit inherits from Asset and tracks explicit combat states (`attackPower`). From this base, units specialize into:

- `MeleeUnit`: Tracks if it serves the frontline directly. Further branches into `FootSoldier` and `Knight` (which uniquely adds a `shieldBlock` state).
- `MountedUnit`: Tracks offensive and movement factors (`chargeMultiplier`, `terrainPenalty`), derived natively into `Cavalry`.

The Siege Engines (`SiegeEngine`) Siege weapons descend from the Asset branch — not from `MilitaryUnit`. A base `SiegeEngine` class inherits from `Asset` and adds `siegeDamage` and `speed`. From this base:

- `BatteringRam`: A slow, heavily armored wall-breaker inheriting from `SiegeEngine`.
- `Catapult`: An area-of-effect weapon inheriting from `SiegeEngine`, adding `aoeDamage` and `aoeTargets`.

The Navy (`Transport`) A `Transport` class inherits from `Asset`, adding `troopCapacity` and `knotsSpeed` to model oceanic movement.

The Nobility (`Noble`) A sprawling lineage tracking complex aristocratic relations (`realmIdx`, `ambitionStat`, `diplomacyStat`, `healthMeter`), as well as relational bindings for lineage tracking (`fatherId`, an array of `childrenIds`, and `childCount`).

The `healthMeter` attribute is initialized to 0.0 for all Noble derivatives and is not specified in the input file.

- `LandedLord`: Rulers overseeing domains, uniquely tracking a `strategyStat` used natively to augment entire armies and defend against shadows.
- `LandlessLord`: Scheming courtiers tracking a continuous `assassinationPlotProgress` accumulator to monitor their tension against the throne.

The Council (`Sage` & `Assassin`) Non-military entities descending directly from Person. Sages track `healingPower` and `wisdom` to mend troops, while Assassins rely on purely isolated mechanics (`stealthStat` and tracking their `clientIdx` who hired them).

Hybrid Lineages (Multiple Inheritance) Occasionally, the world yields entities traversing multiple domains simultaneously.

- `WarriorLord`: Inherits from both `Noble` and `Knight`, possessing the political capital of a Noble and the frontline resilience of a Knight. You must mathematically instantiate all WarriorLords with a baseline 100 HP and 60 Attack Power. They naturally adopt the standard Armor and Maintenance Cost of their Knight base class.
- `WarShip`: Inherits from both `Transport` and `SiegeEngine`, combining oceanic transport capacity with siege firepower. You must explicitly disambiguate shared members (e.g., providing `takeHullDamage()` and `takeWeaponDamage()` methods that delegate to the respective base paths).

You are strictly required to define these complex architectures and gracefully resolve the inevitable compilation ambiguities their multi-lineage definitions create.

2.3 The Realms and Their Holdings (Kingdom Archetypes and Static Binding)

Ten ancient houses partition the continent. There are the frozen wastes claimed by the lords of the Frost Peaks, the bountiful harvests hoarded by the Verdant Expanse, and the maritime empires commanding the Coastal Kingdoms. These lords do not truly own their subjects; they merely point commands at them. The lives of men belong to the continent itself, but their loyalty is pledged to the crown.

There are ten discrete realms explicitly defined within the simulation's geopolitical scope. You must rigorously capture their differences via deep structural inheritance descending from the foundational Kingdom base class.

The Regional Archetypes and Sub-Realms The fundamental Kingdom definition establishes a standard procedural action—collecting taxes and aging subjects—via a foundational performAction() logic block. However, the world inherently fractures into three powerful archetypal lineages before manifesting into the specific ten realms:

1. FrostPeaksKingdom:

- The Realms: `ThePeaksOfFrost`.
- The Mechanics: Tracks a strict `coldAttritionModifier` (default value: 5). When instructed to act, this archetype natively hides the base kingdom behavior, instead executing an `applyColdWeather()` mechanism. This logic traverses its military arrays and inflicts 5 points of flat HP decay (bypassing armor) natively upon all troops before processing standard kingdom duties.

2. VerdantKingdom:

- The Realms: `TheSkyCitadel`, `TheVerdantExpanse`, `TheCrimsonSands`, `TheAzureRiverlands`.
- The Mechanics: Tracks a `foodSurplusBonus`. When instructed to act, this archetype natively hides the base kingdom behavior, executing an `applyFoodBonus()` mechanism. This math natively scales the daily wealth accumulation by adding the `foodSurplusBonus` to the realm's wealth before processing standard kingdom duties.

3. CoastalKingdom:

- The Realms: `TheSunlandEmpire`, `TheCorsairIsles`, `TheTempestCoast`, `TheImperialSeat`.
- The Mechanics: Tracks a `navalTradeBonus`. When instructed to act, this archetype natively hides the base kingdom behavior, executing an `applyTradeBonus()` mechanism. The trade bonus is calculated as `baseTaxIncome * navalTradeBonus * shipCount / 100`, scaling the base tax income proportionally based on the size of the realm's active navy array.

4. Unique Realms (Direct Inheritance):

- `TheVoidRift`: A conceptual entity mimicking a kingdom's structural rules but symbolizing the existential Abyssal Swarm threat. It fundamentally tracks a `threatLevel` rather than wealth or defense, incrementing by exactly +1 per simulated day.

The Execution Matrix The Aethelgard engine class explicitly targets derived kingdom instances natively. The derived realm calculates its unique environmental formulas (like freezing its foot soldiers), then physically chain-calls the base Kingdom execution syntax to securely finalize standard universal tasks (like extracting taxes).

Referential Ledgers The Kingdom class stores basic stats (wealth and defenseStat), but orchestrates actions by traversing a sprawling ledger of purely referential pointers aimed at the overarching engine. Your architecture must define and maintain every tracking array explicitly:

- The Crown: Target pointers linking `currentLord` (a `LandedLord*`), arrays of `heirIds` (used solely for lineage tracking — see note below), and local courtiers (`LandlessLord**`).
- The Vanguard: Structural arrays directly referencing the engine's forces (`FootSoldier**`, `Knight**`, `Cavalry**`, `BatteringRam**`, `Catapult**`, `WarShip**`, `WarriorLord**`).
- The Shadows: Support ledgers referencing `Sage**` members.

Succession Note: The heirIds array exists exclusively for lineage tracking and family tree construction. It has no mechanical effect on succession logic. Succession is always determined by the court member (LandlessLord or WarriorLord) with the highest AmbitionStat, as defined in Section 3 Phase 2.

2.4 The Threads of Destiny (Deterministic Formulas)

To guarantee flawless reproducibility, all systemic actions rely on strictly defined math and threshold accumulators, eliminating chance entirely. Replace any random conceptual elements with the following deterministic structures.

2.4.1 Combat Damage Tracking (The Hourly War Loop)

The daily simulation suspends processing for any two Realms passing the absolute threshold of -80 relations, plunging directly into an hour-by-hour deterministic combat loop strictly governed by unyielding math.

- Effective Base Output: A unit's innate lethality is mathematically augmented by the leading lord. `Effective Damage = BaseAttack * (1.0 + CommandingLord.StrategyStat / 100.0)`. This resultant value serves as raw flat damage sliced directly into enemy HP pools.
- Order of Resolution: Anarchy is strictly disallowed; damages apply in an absolute sequence. Both armies calculate their Cavalry unit damage arrays first. If an enemy's Cavalry array collapses, the remaining damage rigidly spills over directly to the Knight pool, and finally into the FootSoldier pool. Once Cavalry resolution finishes, Knights calculate mathematically, repeating the logic.
- Simultaneous Snapshot Resolution: Within any specific phase (e.g., Cavalry combat), both sides evaluate simultaneously using a snapshot of the current-hour unit counts. Both sides' damage is computed from the pre-phase counts, then casualties are applied simultaneously after all calculations complete. If Unit A deals fatal damage to Unit B within that hour's loop, Unit B's outgoing damage calculation must still register and hit Unit A before Unit B is subsequently excised from the memory ledger at the end of the hour.

2.4.2 Succession and Civil War (Tension Accumulator)

Realms track a persistent `civilWarTension` pool (bounded initially from 0.0 to 100.0) measuring insurrection sentiment.

Tension Modifiers (Daily Ticks):

- The absence of a `currentLord` injects +30.0 daily into the pool.
- Economic insolvency (Wealth < 0) compounds at +5.0 daily.
- Plotting `LandlessLord` subjects natively possessing an `AmbitionStat` greater than 50 inject tension daily equal to `(AmbitionStat / 10.0)`.
- Rulers possessing overwhelming administrative logic (`StrategyStat > 70`) suppress tension by -3.0 daily. Colossal army sizes (>200 units) suppress tension by -1.0 daily.

The Faction Split (100.0 Threshold): Upon bridging the 100.0 boundary, the `LandlessLord` harboring the sheer highest ambition within the array explicitly triggers open revolt (Tie-breaker falling natively to the alphabetically-lowest Name). To manifest this rebellion natively in memory, the engine instantiates a new Realm object using a Copy Constructor that clones the parent's scalar state (halved wealth, halved tax income, halved defenseStat) and allocates empty referential arrays. The engine then sequentially iterates over the parent's military arrays, redistributing precisely half of the reference pointers (slicing n/2 from the tail of each array) into the new rebel faction's arrays managed natively inside the `Aethelgard::rebelFactions**` matrix. No entity objects are copied — only the unowned

reference pointers are transferred. These rebel factions participate in all daily simulation phases following their creation. The parent realm's tension pool then zeroes out implicitly.

2.4.3 Assassination Mechanics (The Internal Coup)

Landless courtiers whose `AmbitionStat` inherently surpasses 60 actively track localized `assassinationPlotProgress` accumulators. This mechanic governs a plotter's attempt to overthrow their own realm's ruler.

The Trajectory of Conspiracy: The plotting metric ticks upward daily calculated as `(AmbitionStat - 60) / 5.0`. If the plotter's realm has any negative diplomatic relation (below 0) with another realm in the relations matrix, the plotting trajectory compounds by +2.0 daily. However, mathematically competent rulers (`StrategyStat > 50`) stunt progress natively by `-(StrategyStat / 25.0)` daily, as do strictly garrisoned keeps (`DefenseStat > 50` subtracting -1.0 daily). The net daily tick is floored at zero (negative ticks do not reduce existing progress).

The Strike (100.0 Threshold): Bridging the 100.0 apex initiates execution strictly calculated by a definitive inequality:

- Calculate `Assassin Power = Plotter.AmbitionStat + (Realm.Wealth / 100)`.
- Calculate `Target Defense = Ruler.StrategyStat + (Realm.DefenseStat / 10) + AgeModifier` (Where a ruler under 40 years is vigilant and inherently gains a +5 modifier, while a ruler over 40 loses -5).
- If the exact Power calculation inherently exceeds Defense, the engine immediately terminates the target entity from memory and nulls pointers naturally provoking succession execution. Otherwise, the plotter entity itself faces immediate native termination, the plotter's realm suffers a flat -30 diplomatic penalty with all other realms, and the plot accumulator zeroes out.

2.4.4 Assassination Mechanics (The Foreign Strike)

Separately from the internal coup, the simulation tracks hired Assassin agents who operate across realm boundaries. These agents target foreign rulers, not their own.

Activation Gate: An Assassin is activated when a plot in their client realm reaches 100.0 AND the client realm's worst diplomatic relation is <= -50. The Assassin then locates the client's worst geopolitical rival (the realm R where `Relations[Client][R]` is lowest).

The Strike Inequality: If `Assassin.StealthStat > TargetLord.StrategyStat`, the target is murdered and succession triggers in the victim's realm. If it fails, the target realm suffers a -20 diplomatic opinion penalty toward the client realm (i.e., `Relations[Target][Client] -= 20`).

- Expendability: Regardless of success or failure, the Assassin object is permanently spent and slated for deletion from memory.

2.4.5 The Final Arbiters (Seeded Tie-Breakers)

In the event of an absolute mathematical tie (e.g., simultaneous entity deaths, or two courtiers possessing the exact same ambition during a civil war), the system cascades through strict indices to guarantee a unique winner:

1. Lower Entity ID wins.
2. Alphabetical comparison of Entity Names.
3. The entity belonging to the lower Realm Index.

3. The Simulation Flow (Daily Cycle)

The simulation executes a strict, continuous daily loop. The simulation runs for exactly `ThreatMax` days (as defined in the input file's `[CONFIG]` section), since `TheVoidRift` increments its `threatLevel` by +1 per day, and the endgame fires when `threatLevel >= ThreatMax`. The simulation may also end early if the endgame resolves into total defeat. Anarchy is banned; the master Aethelgard engine must execute the following 6 absolute phases in strict chronological order.

Phase 1: The Environment & Economic Dispatch

The engine iterates over its specific realm pointers, invoking their statically bound actions (e.g., `applyColdWeather()`). Then, the realm resolves its economy:

- The Formula: `Realm.Wealth = Realm.Wealth + BaseTaxIncome - SUM(UnitCount * MaintenanceCost)`.
- The Bankruptcy Attrition: If `Realm.Wealth < 0`, the engine punishes the realm by clamping the wealth instantly to 0 and explicitly invoking a `takeCasualties()` operation against the FootSoldier array.
- The Casualty Math: `FootSoldierLoss = floor(FootSoldierCount * 0.05)`. (Exactly 5% of foot soldiers vanish).

Phase 2: Aging & Mortality Updates

The engine iterates over the `LandedLord` and `LandlessLord` arrays.

- The Math: Every living lord gains +1 age.
- Health Decay: If a `LandedLord`'s `Age >= 70`, then the script injects `HealthMeter = HealthMeter + 30.0` daily.
- Mortality Trigger: If `HealthMeter >= 100.0`, the engine instantly triggers the lord's `kill()` mechanism and resolves succession natively, promoting the court member (LandlessLord or WarriorLord) with the highest AmbitionStat to ruler. The heirIds array is not consulted. (Note: You must construct an architecture to securely swap the parent Realm's currentLord pointer without crashing the static execution memory—we leave this architectural puzzle to you).

Phase 3: The Shadow Triggers (Plots & Assassins)

The simulation evaluates the accumulators from Section 2.4. This phase contains three distinct sub-stages executed in order:

3a. Civil War Tension (Section 2.4.2) The engine evaluates civil war tension modifiers for each realm per the full tension modifier chain defined in Section 2.4.2. If tension reaches 100.0, a faction split is triggered.

3b. The Internal Coup (Section 2.4.3) For every `LandlessLord` where `AmbitionStat > 60`, their PlotProgress ticks daily per the full Section 2.4.3 modifier chain. When a plot reaches 100.0, the strike resolves using the AssassinPower vs TargetDefense inequality defined in Section 2.4.3.

3c. The Foreign Strike (Section 2.4.4) When a plot in an Assassin's client realm reaches 100.0 AND the worst diplomatic relation is <= -50, the active Assassin locates the client's worst geopolitical rival. The strike resolves using the `StealthStat > StrategyStat` inequality defined in Section 2.4.4.

Phase 4: Geopolitical Decay

The engine flattens all diplomatic momentum by iterating over the 10x10 relations matrix.

- The Entropy Formula: For every relationship entry [i][j] (where i != j), if `Relations > -100`, then `Relations = Relations - 1`.

Phase 5: Open Warfare Validation

The engine sweeps the diplomacy matrix. If any `Relations[i][j] <= -80`, war is declared violently and the daily tick suspends.

- Aggressor Tie-Breaker: The attacking realm is precisely the one whose LandedLord possesses the strictly higher AmbitionStat (falling back to array index order on absolute ties).
- 0 Power Abort: If the definitively chosen attacker possesses exactly 0 total Military Power, they are incapable of mounting an invasion. The war is aborted immediately, and to prevent a stagnant grudge, Relations[Atk][Def] and Relations[Def][Atk] are both reset to 0.
- Otherwise, the engine initiates the Hourly War Loop detailed in Section 2.4.1.

Phase 6: The Void Rift Escalation (Endgame Thresholds)

The existential void rift entity increments its native `threatLevel` by exactly +1. If `threatLevel >= ThreatMax` (loaded from the input file's `[CONFIG]` section), the endgame runs a global calculation to determine Aethelgard's fate.

- Power Weighting Script: `Power = (FootSoldiers * 1) + (Cavalry * 5) + (Knights * 8) + (Ships * 12) + (Rams * 25) + (WarriorLords * 10)`.
- The Swarm Threat: The abyssal force equates exactly to `threatLevel * 100` power.
- The Cost of Victory: If `GrandArmy.Power >= Threat.Power`, Aethelgard miraculously survives, but suffers a blanket 60% flat casualty rate across all military arrays (`UnitCount_Remaining = floor(UnitCount * 0.4)`). Otherwise, the simulation terminates the entire program structure gracefully in absolute defeat.

4. Input & Expected Output

Your engine must ingest the starting state via a unified CSV file parsing system. You will be provided with two scenarios:

1. scenario_alpha.csv
2. scenario_omega.csv

4.1 Simulation Data Structure (Unified CSV)

Section	Purpose	Attributes
[KINGDOMS]	Defines the 10 realms (Indices 0–9).	RealmID, Name, Wealth, TaxIncome, DefenseStat, Terrain
[RELATIONS]	Initial diplomatic standings.	A 10x10 matrix of space-separated integers
[LORDS]	Rulers and courtiers (without spouse data).	LordID, Name, Age, Gender, Class, RealmIdx, Strategy, Ambition, Diplomacy, FatherID
[MILITARY]	Global army composition and unit stats.	RealmIdx, UnitType, Count, Attack, HP, SiegeDmg, AoE

[SAGES]	Non-military support characters.	RealmIdx, Name, HealingPower, Wisdom
[ASSASSINS]	Hired shadow agents.	AssassinID, Name, Age, StealthStat, ClientRealmIdx
[CONFIG]	Global simulation parameters.	ThreatMax

4.2 Expected Output (The Royal Dashboard)

While you will not be graded on exact string formatting, your simulation must provide the following information during each daily tick:

4.2.1 The Scholar's Dashboard (Daily Snapshot)

A comprehensive summary printed at the start of every day:

- The Realm Ledger: A table listing all active realms, their Wealth, Total Military Power, and the Current Lord.
- The Diplomatic Web: A labeled 10x10 matrix of relational values.
- The Whispering Council: A list of any `LandlessLord` with `PlotProgress > 50%` and all active Assassin entities.
- The Chronometer: The current Day and Invasion Progress (`threatLevel / threatMax`).

4.2.2 The War Room (Combat Resolution)

When war is declared (Section 3 Phase 5), output a detailed Battle Report:

- The Proclamation: Display the identities of the Attacker and Defender.
- The Hourly Sequence: For each hour of battle, log damage phases: `[SIEGE]`, `[FLANK]`, `[ELITE]`, `[LINE]`, and `[HEAL]`.
- The Conclusion: Report casualties and mark the outcome: `>>> CONQUERED <<<`, `>>> ROUTED <<<`, or `>>> STALEMATE <<<`.

4.2.3 The Chronicler's Chronicle (Global Events)

Log deterministic events as they occur using bracketed tags:

- Estate Shifts: `[DEATH]` and `[SUCCESSION]` notices.
- Subterfuge: `[PLOT]` warnings and `[ASSASSINATION]` strike reports.
- The Doom: `[THREAT]` increases and the final `[ABYSSAL ECLIPSE]` result.

5. Architectural Mandates (The Zero-Tolerance Constraints)

To ensure you confront raw memory architectures directly, the following absolute constraints are enforced globally. Violating any of these results in an instant zero.

- NO Standard Template Library (STL): Vectors, maps, smart pointers, etc., are explicitly banned. You must manage sizes, resizing, and array structures using dynamically allocated raw pointer arrays natively (e.g., `Lord**`).

- NO Polymorphism or Virtual Functions: Dynamic dispatch is strictly forbidden. Base class pointers cannot orchestrate derived behaviors. All inheritance operates fundamentally to reuse structural data, but methods must be hidden and bound natively using explicit types.
- Strict Memory Ownership: Localized Kingdom objects cannot possess ownership (new or delete privileges) over entities. The central Aethelgard engine class exclusively allocates, manages, and destroys all entity memory arrays globally. Kingdom arrays are purely unowned referential pointers.

6. Deliverables & Grading

UML Class Diagram : **40%** of the total assignment marks are strictly reserved for your UML class diagram. You will be provided with a formal grading rubric and a self-evaluation sheet, both of which will be used to assess your architecture during demo time

Appendix A: The Master Architect's Ledgers (Comprehensive Formulas)

The following reference tables define the absolute, bare-metal algebraic calculations your engine must implement to guarantee determinism. No deviations are permitted. These tables consolidate all formulas from Section 2 and Section 3 into a single quick-reference.

Table I. Environmental & Demographic Processing

Execution Domain	Formula	Required Trigger Condition
Gross Wealth	Wealth + BaseTaxIncome	Daily.
Net Wealth	GrossWealth - SUM(UnitCount * MaintenanceCost)	Daily.
Bankruptcy Penalty	FootSoldierLoss = floor(FootSoldierCount * 0.05)	Fires instantly if Net Wealth < 0. Wealth is clamped to 0.
Mortal Aging	Age = Age + 1	Applies to all living LandedLord and LandlessLord entities daily.
Geriatric Decay	HealthMeter = HealthMeter + 30.0	Fires daily strictly for LandedLords where Age >= 70.
Natural Death	kill() and invoke resolveSuccession()	Triggers instantly when HealthMeter >= 100.0.

Table II. Subterfuge & Diplomacy

Execution Domain	Formula	Required Trigger Condition

Plot Escalation	PlotProgress += (AmbitionStat - 60) / 5.0 (+ modifiers per Section 2.4.3)	Daily for LandlessLords where AmbitionStat > 60. Net tick floored at 0.
Internal Coup Strike	AssassinPower = Plotter.AmbitionStat + (Realm.Wealth / 100) vs TargetDefense = Ruler.StrategyStat + (Realm.DefenseStat / 10) + AgeModifier	Fires when a LandlessLord's plot reaches 100.0. Success kills own ruler. Failure kills the plotter; plotter's realm loses 30 relations with all other realms. (Section 2.4.3)
Foreign Assassin	Target Realm R where Relations[Client][R] is lowest.	Fires when a plot reaches 100.0 AND worst relation is <= -50. (Section 2.4.4)
Foreign Strike Math	Assassin.StealthStat > Target.StrategyStat	Success kills the foreign target. Failure invokes a -20 relations penalty (Relations[Target][Client] -= 20). The Assassin object is always deleted post-strike. (Section 2.4.4)
Geopolitical Decay	Relations[i][j] -= 1	Applies to all matrix cells where i != j and Relations > -100.
War Threshold	Attacker = Realm whose LandedLord possesses the definitively higher AmbitionStat. Falls back to lower array index on ties.	Triggers when Relations[i][j] <= -80.

Table III. The Hourly War Loop

Execution Domain	Formula	Required Sub-Phase
Strategy Multiplier	Modifier(M) = 1.0 + (CommandingLord.StrategyStat / 100.0)	Calculated once per hour of battle.
Siege Phase	Damage = SUM(RamDmg + CatapultDmg + ShipSiegeDmg) * M	Hits walls conceptually; opens the path to the main armies.
Flanking Phase	Damage = SUM(CavAttack * CavChargeMultiplier) * M	Cavalry strikes.
Elite Phase	Damage = (SUM(KnightAttack) + SUM(WarriorLordAttack)) * M	Elite strikes. A WarriorLord's attackPower is its Knight base class value. M applies to the total elite sum.

Infantry Phase	Damage = (FootCount * 10) * M	The frontline meat grinder.
War Attrition	Opponent takes casualties: floor(IncomingDamage * 0.05)	Applied cross-pool natively per hour.
Naval Attrition	ShipLoss = MAX(1, floor(ShipCount * 0.1)) if ShipCount > 0, else 0	Ships sink at a flat 10% rate per hour of combat, with a minimum of 1.
Triage (Healing)	RestoredFoot = MIN(SUM(SageHealing), floor(IncomingInfantryDamage * 0.01))	Sages restore foot soldier headcount by re-activating incapacitated units. No new FootSoldier objects are allocated; the count is directly incremented by RestoredFoot per hour.
Morale Collapse	End war if AttackerPower < (DefenderPower * 0.5)	The routing threshold. The conqueror severs 75% of the loser's Wealth.
Stalemate	Triggered if hour 24 finishes without Morale Collapse. Relations[Atk][Def]=0 and Relations[Def][Atk]=0.	The armies are exhausted.
War Aftermath	Relations[Atk][Def]=0, Relations[Def][Atk]=0 (Attacker Routs) OR [Atk][Def]=0, [Def][Atk]=-50 (Defender Breaks).	Applies immediately following Morale Collapse. All other realms decrement Relations[X][Attacker] by 2.

Table IV. The Void Rift (Global Check)

Execution Domain	Formula	Required Trigger Condition
Global Alliance Power	SUM(Foot) + SUM(Cav*5) + SUM(Knights*8) + SUM(Ships*12) + SUM(Rams*25) + SUM(WarriorLords*10)	Triggers when TheVoidRift.ThreatLevel >= ThreatMax.
The Swarm Threshold	AbyssalPower = ThreatLevel * 100	The math required to avoid absolute wipeout.
The Cost of Victory	UnitCount_Remaining = floor(UnitCount * 0.4)	A catastrophic 60% loss imposed across all survival forces globally (FootLoss = Count * 0.6).

Q3: Lazarus Machinae (200 Marks)

Prologue

Deep within the archives of FAST, beneath decades of deprecated code, a massive discovery was made. A preservation team known as Lazarus Machinae was formed a few years back and is composed of students from FAST and this team is tasked with preserving the lost architectures of the inception of computers. This team was tasked with the preservation of this newly found architecture named Onyx-16.

Alongside these schematics, the research team uncovered the language and the compiler used with this architecture. This language was Turing Complete.

There is only one problem. The physical hardware is extinct. There is no physical machine left on Earth that can execute programs written in this language. We have the software but the body it lived in is dead.

The Mission

You have been drafted into the team and your mission is the resurrection of this architecture. You're not writing the compiler nor writing programs in this old language. You must reconstruct the entire architecture from the docs so that you can run these uncovered, ancient programs.

The catch is that this architecture will be mimicked in an emulator / virtual machine that you'll be designing.

The Docs

Chapter 1: System Architecture

1.1 Overview

The ONYX-16 is a 16-bit word machine with byte-level addressing. Every memory address in the system points to a single 8-bit byte. Instructions are 16 bits wide and therefore require two consecutive single-byte bus transactions to load from memory — a high byte fetch followed immediately by a low byte fetch.

All arithmetic, all register storage, and all memory addresses operate on a maximum word width of 16 bits (0x0000 to 0xFFFF).

1.2 Required Components

A functional Onyx-16 system requires the following components:

Sr #	Unit	Responsibility
1	Mainboard	Owns the signal pathways; routes every bus transaction; the chassis all other units plug into
2	Processor	Fetches instructions from memory, decodes them, and drives the execution datapath
3	Memory Module	Stores the loaded program binary and all runtime data

4	Graphics Adapter	Receives output byte payloads from the Mainboard's address decoder and forwards them to the display
5	Phosphor Display	Buffers rendered characters and flushes the full output to screen at program end
6	Keyboard	Maintains a live input buffer; blocks and waits for user input when the buffer runs dry

1.2 The Mainboard as the Central Authority

No unit in this system ever communicates directly with another unit. The Processor does not speak to the Memory Module. The Processor does not speak to the Keyboard. The Graphics Adapter does not speak to the Memory Module.

Every interaction between units is mediated exclusively by the Mainboard. A unit places a value on a pathway pin, asserts a signal line, and triggers a single Mainboard clock pulse. The Mainboard reads the address, decides the destination, completes the transaction, and de-asserts the signal line. This is the only mechanism of inter-unit communication in the entire system.

Chapter 2: The Registers

2.1 Overview

The Processor contains a single internal Register Bank. It is a monolithic sealed unit — it is not a collection of loose components. It houses eleven individually sealed storage cells, each accessible only through a defined interface. No circuit outside the Storage Bank, including the Processor itself, may observe or modify the internal charge state of any cell by any means other than those signal lines.

2.2 Storage Cells

Sr#	Width	Purpose
R0	16-bit	General Purpose Register
R1	16-bit	General Purpose Register
R2	16-bit	General Purpose Register
R3	16-bit	General Purpose Register
R4	16-bit	General Purpose Register
R5	16-bit	General Purpose Register
R6	16-bit	General Purpose Register
R7	16-bit	General Purpose Register
PC	16-bit	Program Counter — the byte address of the next instruction to fetch

IR	16-bit	Instruction Register — holds the currently executing 16-bit instruction word
FLAGS	8-bit	Status Latch — encodes the outcome of the last comparison or arithmetic operation

Initialization: All the registers must be initialized to 0x0000 state at startup.

2.3 Access Constraint

All these registers must only be accessible to the outside using 2 ways:

- READ: Returns the registers current value
- WRITE: Writes the passed value to a register

There is no other way of editing or accessing the register bank.

2.4 Status Flags register

The `FLAGS` cell is 8 bits wide. Only four of those bits carry architectural meaning. The remaining bits must be preserved exactly as-is across all operations — they must never be written to or cleared by any instruction logic.

Bit	Mask	Name	Set When
0	0x01	Zero Flag (ZF)	A comparison found both operands equal, or an arithmetic result was exactly 0x0000
1	0x02	Negative Flag (NF)	A comparison found the left operand strictly less than the right operand
2	0x04	Positive Flag (PF)	A comparison found the left operand strictly greater than the right operand
7	0x80	Halt Flag (HF)	A thermal shutdown event occurred — set once, never cleared

Mutual exclusion rule: ZF, NF, and PF are mutually exclusive at all times. Setting any one of them must simultaneously clear the other two. No instruction may leave two of these three bits set at once.

HF behavior: Once bit 7 is set, the Processor permanently stops executing instructions. This flag cannot be cleared by software. It is the terminal state of the machine.

Chapter 3: Arithmetic Logic Unit

This is the circuit responsible for all the computation. It is a stateless block and it has no memory of past invocations.

3.1 Interface

The ALU receives the following inputs upon invocation:

1. An operator selector (OPCODE in the instruction)

2. Operand A: a 16-bit value in the destination register.
3. Operand B: a 16-bit value in the source register.
4. A direct connection point to the FLAGS register.

The ALU provides only one output:

- A 16-bit result value - It's written back to the destination register.

The FLAGS registers update is never outputted. The ALU has a permanent connection to it and performs updates on it upon every execution. The invoking point of the ALU just checks the FLAGS register after execution to see if it's updated.

3.2 Operation Table

Operation	Input Used	Result written to Register?	FLAGS updated
ADD	A + B	YES	ZF only
SUB	A - B	YES	ZF only
MUL	A * B	YES	ZF only
DIV	A / B	YES	ZF only
AND	A & B	YES	ZF only
OR	A | B	YES	ZF only
XOR	A ^ B	YES	ZF only
NOT	~A	YES	ZF only
INC	A + 1	YES	ZF only
DEC	A - 1	YES	ZF only
CMP	A vs B	NO	ZF, PF, NF

DIV BY ZERO: If Operand B is zero, the hardware does not fault. The result is silently forced to 0x0000.

Overflow: All arithmetic is unsigned 16-bit. Results wrap silently on overflow.

ZF update rule for arithmetic operations: After any non-CMP operation, if the result word is 0x0000, ZF is set. Otherwise, ZF is cleared.

CMP flag update rules:

- If A == B → Set ZF, clear NF and PF.
- If A < B → Set NF, clear ZF and PF.
- If A > B → Set PF, clear ZF and NF.

CMP does not produce a numeric result. No register writeback occurs.

Chapter 4: The System Buses

The ONYX-16 mainboard contains 3 system buses. These buses are the conduits through which every memory transaction takes place. As these are copper pathways etched onto the motherboard itself, if the motherboard is destroyed these buses are destroyed with it.

Bus	Width	Function
Address	16-bit	Carries the target memory address for the pending function
Data	8-bit	Carries the data being read from or written to memory
Control	2 signal values	READ ENABLE or WRITE ENABLE - Only one can be asserted at a time

4.1 Bus Transaction Protocol

Every memory access follows this exact sequence:

For a READ transaction:

1. Write the target address to the Address Pathway.
2. Assert the READ ENABLE line on the Control Pathway.
3. The mainboard completes routing and places the retrieved byte onto the Data Pathway.
4. The READ ENABLE line is de-asserted automatically by the mainboard after the transaction.

For a WRITE transaction:

1. Write the target address to the Address Pathway.
2. Write the data byte to the Data Pathway.
3. Assert the WRITE ENABLE line on the Control Pathway.
4. The mainboard routes the byte to the target destination.
5. The WRITE ENABLE line is de-asserted automatically by the mainboard after the transaction.

Only one transaction may occur per clock pulse. The pathways are not shared concurrently. A second transaction must wait for the first to complete.

Chapter 5: The Memory Module

The ONYX-16 uses a single-flat memory module. It's a separately manufactured unit that plugs into the motherboard in a dedicated slot.

Property	Value
Total Capacity	3,840 bytes
Valid Address Range	0x0000 – 0x0EFF (inclusive)
Cell Width	8 bits (byte-addressed)
Power-On State	All cells initialized to 0x00

Out-of-Bounds Access: Any read or write targeting an address of `0x0F00` or higher constitutes a Segmentation Fault. On an invalid read, the Data Pathway floats to `0xFF`. On an invalid write, the data is discarded. In both cases, a hardware fault diagnostic message is emitted.

The Memory Module does not implement the MMIO address intercept. It only ever sees addresses it is legally authorized to service. The routing decision — whether a given address reaches the Memory Module or not — is made exclusively by the mainboard's address decoder (see Chapter 6).

Chapter 6: The Address Map and MMIO Decoder

The mainboard contains a hardware address decoder that inspects every bus transaction before forwarding it. The complete physical address map of the ONYX-16 is as follows:

Address Range	Size	Region	Routed To
0x0000 – 0x07FF	2,048 bytes	Code Segment	Memory Module
0x0800 – 0x0EFF	1,792 bytes	Data Segment	Memory Module
0x0F00 – 0x0FEF	—	Fault Zone	Segmentation Fault — no valid device
0x0FF0	1 byte	Keyboard Character Port	I/O Panel — READ only
0x0FF1	1 byte	Display Character Port	Graphics Adapter — WRITE only
0x0FF2	1 byte	Display Integer Port	Graphics Adapter — WRITE only
0x0FF3	1 byte	Keyboard Integer Port	I/O Panel — READ only

6.1 MMIO Interception Rule

When the address decoder detects a transaction targeting addresses `0x0FF0` through `0x0FF3`, it intercepts the transaction before it reaches the Memory Module. The Memory Module never sees these addresses. The mainboard redirects the transaction directly to the appropriate peripheral hardware.

This interception is performed inside the mainboard itself. No external circuit, no Processor logic, and no Memory Module logic participates in this decision. The mainboard is the sole authority.

6.2 Peripheral Port Contracts

Port	Direction	Contract

0x0FF0	READ	Returns the ASCII byte value of the next character in the keyboard input buffer. If the buffer is empty, the mainboard suspends the clock and waits for the user to provide input before completing the transaction.
0x0FF1	WRITE	The written byte is treated as an ASCII character code and forwarded directly to the display for rendering.
0x0FF2	WRITE	The written byte is treated as an unsigned 8-bit integer. The mainboard converts it to its decimal digit string and writes each ASCII character to the display sequentially. Example: writing 0x2A (decimal 42) renders the characters '4' then '2'.
0x0FF3	READ	Reads and parses a complete decimal integer token from the keyboard buffer. Returns the parsed value as an unsigned 8-bit result. If the buffer is empty, the mainboard suspends the clock and waits for input.

Chapter 7: Peripheral Units

7.1 The I/O Panel (Keyboard)

The keyboard is an external component attached to the physical chassis. It maintains an internal First In First Out buffer. If prompted and the buffer is empty the keyboard invokes a [Hardware Interrupt] and blocks until the user inputs something. This input is then enqueued into the buffer character by character followed by a space delimiter.

Electrical interface exposed to the mainboard:

Signal	Direction	Behavior
PENDING_INPUT	Output pin	Driven HIGH when the buffer contains at least one non-whitespace character. The mainboard reads this pin before deciding whether to suspend execution.
READ_CHAR	Triggered by mainboard	Pops and returns the next non-whitespace byte from the buffer as an 8-bit ASCII value.
READ_INT	Triggered by mainboard	Parses and consumes the next complete decimal integer token from the buffer. Returns an 8-bit unsigned value. Leading and trailing whitespace is discarded automatically.

7.2 The Graphics Adapter

The Graphics Adapter is a separate physical card that plugs into the mainboard's expansion slot. It receives raw byte payloads from the mainboard's MMIO interceptor and translates them for the connected display.

The Graphics Adapter requires a Display Unit to be physically connected to its output port. If no display is connected, all received payloads are silently discarded.

Electrical interface exposed to the mainboard:

Signal	Triggered By	Behavior
CHAR_PAYLOAD	Write to 0x0FF1	Forwards the received byte directly to the display as a single ASCII character.
INT_PAYLOAD	Write to 0x0FF2	Converts the received byte to a decimal digit string and sends each character to the display sequentially.

7.3 The Phosphorus Display

The Phosphor Display is a 32-column × 16-row CRT screen. It maintains an internal string buffer. Characters accumulate in the buffer as they arrive from the Graphics Adapter during program execution.

The buffer is not flushed to the screen automatically during execution. The display renders its entire buffer exactly once — at the explicit conclusion of program execution — inside a bordered phosphor-green terminal frame. Lines longer than 32 characters wrap to the next row. Rows beyond 16 are clipped.

Chapter 8: The Processor

The Processor is a single-chip unit. It contains four internal sub-units:

Internal Sub-Unit	Role
Storage Bank	Holds all registers (Chapter 2)
Arithmetic Execution Core	Performs all computation (Chapter 3)
Block Cache	16-byte high-speed prefetch buffer (Section 8.1)
Instruction Decode Matrix	Decodes instructions and drives the execution datapath (Section 8.2)

8.1 Physical Bus Dependency

The Processor has three physical connector ports on its underside that mate with the three system pathways on the mainboard. These ports carry no voltage until the Processor is physically seated in a mainboard socket. Without a connected mainboard:

- The Processor cannot read from or write to memory.
- The Processor cannot receive input or produce output.
- The Processor cannot execute any instruction.

When the Processor is seated, the mainboard drives the Address, Data, and Control pathway pins into the Processor's connector ports. The Processor then holds a live reference to all three pathways for the duration of its installation. The Processor does not own these pathways — they belong to the mainboard and continue to exist if the Processor is removed.

8.2 The Block Cache

Embedded in the Processor die is a 16-byte block cache. It mirrors a single contiguous 16-byte block from main memory to reduce repeated bus transactions during instruction fetching.

Property	Value
Block size	16 bytes
Number of blocks	1 (single-block, direct-mapped)
Base address register	Records the starting address of the block currently loaded
Valid bit	A single flag indicating whether the cache currently holds valid data

Cache hit: A requested byte address falls within [base, base + 16] and the valid bit is set. The byte is returned directly from the cache without a bus transaction.

Cache miss: Either the valid bit is not set, or the requested address falls outside the cached block. The Processor issues a bus read transaction to the mainboard, retrieves the byte, and the cache is updated with a fresh 16-byte block starting at the aligned boundary.

Cache invalidation: Any write transaction to an address within the currently cached block immediately clears the valid bit.

8.3 The Instruction Decode Matrix

The Instruction Decode Matrix is a fixed 256-entry lookup table burned into the Processor at manufacture time. It is indexed by the 8-bit opcode field of an instruction. Each entry in the table is a signal vector — a collection of boolean and selector flags that dictates exactly how the execution datapath should behave for that instruction.

This matrix is embedded into the control unit which is inside the Process besides the ALU.

The signal vector for any given opcode specifies:

Signal Type	Meaning
IS_VALID	bool — Whether this opcode is recognized. Invalid opcodes emit a hardware fault and abort the cycle.
USES_AEC	bool — Whether the Arithmetic Execution Core should be invoked.
AEC_OPERATION	enum — Which operation the AEC should perform (ADD, SUB, MUL, etc.).
IS_MEMORY_READ	bool — Whether the cycle should read a byte from memory via the bus.
IS_MEMORY_WRITE	bool — Whether the cycle should write a byte to memory via the bus.

IS_BRANCH	bool — Whether the PC should be conditionally or unconditionally redirected.
BRANCH_ON_ZERO	bool — If branching, only branch when ZF is set.
BRANCH_ON_NOT_ZERO	bool — If branching, only branch when ZF is clear.
WRITES_TO_REGISTER	bool — Whether the result of this cycle should be written back to the destination register.
IS_4BIT_IMMEDIATE	bool — Whether the operand is a 4-bit immediate packed into the instruction word (Format B).
IS_16BIT_IMMEDIATE	bool — Whether the operand is a 16-bit immediate requiring a second word fetch (Format D).

The Decode Matrix eliminates all branching logic from the execution engine. The execution datapath reads the signal vector and mechanically follows its flags — it never inspects the opcode itself. All semantic intelligence is pre-burned into the table at construction time.

8.4 The Fetch-Decode-Execute Cycle

Each clock tick drives the Processor through exactly one complete instruction cycle:

Step 1 — FETCH (Two bus reads):

1. Place the current value of PC on the Address Pathway. Assert READ ENABLE. Receive the high byte from the Data Pathway.
2. Place PC + 1 on the Address Pathway. Assert READ ENABLE. Receive the low byte from the Data Pathway.
3. Assemble the 16-bit instruction word: instruction = (highByte << 8) | lowByte.
4. Write the assembled word into the IR register.

Step 2 — INCREMENT PC:

Advance PC by 2 (one instruction word = two bytes). This happens before decode so that any branch instruction in the decode phase calculates offsets relative to the next instruction, not the current one.

Step 3 — DECODE:

Extract the opcode from bits [15:8] of IR. Look up the signal vector in the Decode Matrix.

If `IS_16BIT_IMMEDIATE` is set (Format D instruction), the Decode Matrix triggers two additional bus reads at the current PC to fetch the 16-bit immediate payload word. PC is then advanced by an additional 2 bytes.

Step 4 — EXECUTE:

The datapath follows the signal vector mechanically:

- If `USES_AEC`: invoke the AEC with the selected operation and operands; update FLAGS via the direct latch connection.
- If `IS_MEMORY_READ`: place the source register's value on the Address Pathway, assert READ ENABLE, receive the byte from the Data Pathway. That byte becomes the execution result.

- If `IS_MEMORY_WRITE`: place the pointer register's value on the Address Pathway, place the source register's value on the Data Pathway, assert WRITE ENABLE.
- If `IS_BRANCH`: check the `BRANCH_ON_ZERO` and `BRANCH_ON_NOT_ZERO` signals against the current ZF. If neither is set, the branch is unconditional. If the condition is satisfied, load the target register's value directly into the PC, overwriting it.
- If `IS_4BIT_IMMEDIATE`: the execution result is the 4-bit immediate value packed into the lower nibble of the instruction word. No bus transaction occurs.
- If `IS_16BIT_IMMEDIATE`: the execution result is the 16-bit value fetched during the decode step. No further bus transaction occurs.

Step 5 — WRITEBACK:

If `WRITES_TO_REGISTER` is set in the signal vector, write the execution result into the destination register (identified by bits [7:4] of the instruction word).

Halt Condition:

Before each cycle begins, the Processor checks the HF bit (`0x80`) of the FLAGS latch. If it is set, the Processor silently refuses to execute and the clock loop terminates.

8.5 Thermodynamics

Unlike software emulators, this will react to physical load via a thermodynamic model:

- Base Temperature: The processor spawns as a base temperature of 25.0 C
- Clock Friction: On every single hardware tick, the current temperature inherently rises by 0.05 C (0.15 C if overclocked)
- Thermal Shutdown: After the completion of one cycle the temperature is checked if it has exceeded 90.0 C and if it has the CPU triggers the physical fault (flags |= 0x80), permanently locking the processor and halting the system.

Chapter 9: The Motherboard / Mainboard

9.1 Component Sockets

The Mainboard exposes the following physical slots. Each slot is either empty or occupied by exactly one component at a time. A component plugged into a slot does not become part of the mainboard — it remains a separate, independently manufactured unit that can be removed without damaging anything else.

Socket	Accepts	Notes
CPU Socket	Processor	Upon insertion, the mainboard immediately drives the Address, Data, and Control pathway pins into the Processor's connector ports.
RAM Slot	Memory Module	The Memory Module receives all standard address-range transactions.
Expansion Slot	Graphics Adapter	Receives MMIO write payloads from the address decoder.

ATX Power Connector	Power Supply Unit	Without a connected PSU, the system has no power and all clock pulses are suppressed.
I/O Panel Connector	Keyboard	Polled by the address decoder on MMIO read transactions.

A socket with nothing installed is a floating pin. If the address decoder routes a transaction to an unoccupied slot, the Data Pathway floats to 0x00 on reads, and writes are silently discarded. No crash occurs.

9.2 The Hardware Clock - pulseClock()

The mainboard exposes a single public signal line: `pulseClock()`. This is the heartbeat of the entire system. Every bus transaction in the entire machine is completed by calling `pulseClock()` exactly once.

No component other than the mainboard performs any routing. No component directly accesses another component. The Processor never speaks to the Memory Module directly. The Processor never speaks to the Keyboard directly. The Processor places values on the pathway pins, asserts a control signal, and calls pulseClock(). The mainboard does everything else.

9.3 The Power Supply Unit

The PSU plugs into the motherboard's ATX connector. It carries a maximum wattage capacity set at manufacturing time. The PSU holds a back-reference to the mainboard it is connected to.

On each clock cycle, the total power draw of active components is polled and passed to the PSU. If the total draw exceeds the PSU's rated capacity, the PSU calls `killPower()` on the mainboard through its stored back-reference. This sets the system power flag to false, which causes all subsequent `pulseClock()` calls to return immediately, permanently halting execution.

The PSU does not own the mainboard. It holds a reference to it. The mainboard exists before the PSU is connected and continues to exist in its current state after the PSU trips. Only the power flag changes.

The individual components keep track of the power they draw at every cycle.

- Memory Module: Draws 0.5 W during active read/write ops. But drops to 0.1 W in idle state.
- Graphics Card: Exerts a massive 15.0 W spike when writing something to the screen and draws 2.0 W in idle state.

Chapter 10: The main.cpp

In the `main.cpp` all the components will be created and inserted where ever they need to be inserted and then the main function will act as the bootloader asking for which script of the custom language to boot from and then pass the name of that file and the ram to the `loadProgramAndFlash()` function in the Interpreter class. This will flash the program into the specific memory addresses. There on out your own logic must be able to run that program's binary using the details above.

APPENDIX A: INSTRUCTION SET REFERENCE

The ONYX-16 ISA contains 13 distinct opcodes. Each instruction accepts operands in one of three encoding formats (see Appendix B). The ISA supports two dialects: the original Urdu Assembly dialect and the preserved English transliteration. Both compile to identical machine code.

Opcode	Urdu Mnemonic	English Mnemonic	Format	Operation Description
0x00	AARAM	NOP	A	No operation. A full instruction word of 0x0000 also signals end-of-program to the bootloader.
0x01	JAMA	ADD	A	R[dest] = R[dest] + R[src]. Updates ZF.
0x02	TAFREEK	SUB	A	R[dest] = R[dest] - R[src]. Updates ZF.
0x03	ZARAB	MUL	A	R[dest] = R[dest] * R[src]. Updates ZF.
0x04	TAQSEEM	DIV	A	R[dest] = R[dest] / R[src]. Updates ZF. Result is 0 if R[src] is zero.
0x0A	MUWAZANA	CMP	A	Compare R[dest] against R[src]. No register writeback. Updates ZF, NF, and PF.
0x10	CHHALANG	JMP	A	PC = R[src]. Unconditional jump. The dest nibble is unused (set to 0).
0x11	AGAR_SIFAR	JZ	A	If ZF = 1: PC = R[src]. Jump if the last comparison was equal. The dest nibble is unused.
0x12	AGAR_MAUJOOD	JNZ	A	If ZF = 0: PC = R[src]. Jump if the last comparison was not equal. The dest nibble is unused.
0x1A	BHARO	LDR_IMM	B	R[dest] = imm4. Loads a 4-bit immediate value (range 0x0 to 0xF). Single-word instruction.
0x1B	BHARO	LDR_IMM	D	R[dest] = imm16. Loads a full 16-bit immediate value. Two-word instruction (see Appendix B). Used for all values above 0x000F and for all label addresses.
0x20	PARHO	LDR	A	R[dest] = Memory[R[src]]. Loads one byte from the address held in R[src] into R[dest].
0x21	RAKHO	STR	A	Memory[R[src]] = R[dest]. Writes the value in R[dest] to the address held in R[src]. The dest nibble carries the value; the src nibble carries the pointer.

Important note on `BHARO / LDR_IMM`: The assembler automatically selects Format B or Format D based on the magnitude of the immediate value. Any immediate value from 0 to 15 uses Format B (single

word). Any value above 15, any hex literal above `0x000F`, and all label references unconditionally use Format D (two words). The programmer does not select the format — the assembler does.

APPENDIX B: INSTRUCTION ENCODING FORMATS

All instructions on the ONYX-16 are encoded in one of three physical formats. The format is determined by the opcode and is pre-defined in the Decode Matrix — it is not programmer-selectable.

Format A — Standard Two-Register (Single Word)

Used by all arithmetic, comparison, memory, and branch instructions.

```
/dev/null/format-a.txt#L1-5
Bit Layout (16 bits):
[ 15 ........... 8 ] [ 7 ... 4 ] [ 3 ... 0 ]
[    OPCODE (8)    ] [ DEST (4)] [ SRC  (4)]

Total size: 2 bytes (1 word)
```

Format B — 4-bit Immediate (Single Word)

Used exclusively by `BHARO / LDR_IMM` when the immediate value fits in 4 bits.

```
/dev/null/format-b.txt#L1-5
Bit Layout (16 bits):
[ 15 ........... 8 ] [ 7 ... 4 ] [ 3 ... 0 ]
[    OPCODE (8)    ] [ DEST (4)] [ IMM4 (4)]

Total size: 2 bytes (1 word)
Opcode: 0x1A
```

Format D — 16-bit Immediate (Two Words)

Used by `BHARO / LDR_IMM` when the immediate value exceeds 4 bits, or when the operand is a label address.

```
/dev/null/format-d.txt#L1-9
Bit Layout — Word 1 (Header, 16 bits):
[ 15 ........... 8 ] [ 7 ... 4 ] [ 3 ... 0 ]
[    OPCODE (8)    ] [ DEST (4)] [ 0000    ]

Bit Layout — Word 2 (Payload, 16 bits):
[ 15 ............ 0 ]
[   IMM16  (16)      ]

Total size: 4 bytes (2 consecutive words)
Opcode: 0x1B
```

Assembler behavior for Format D label resolution: During Pass 1, the assembler tracks the exact byte size of every instruction (including whether a given BHARO will expand to Format B or D). It uses this to calculate the precise byte address of every label. In Pass 2, all label references are substituted with their resolved 16-bit byte addresses and are always encoded as Format D, regardless of the numeric value of the address.

APPENDIX C: THE ONYX-16 ASSEMBLY LANGUAGE

The source language compiles to ONYX-16 machine code via a two-pass assembler. Programs are written in plain text files using either the Urdu dialect or the English dialect (both are recognized interchangeably). The Interpreter and the sample programs to be run on ONYX-16 have been provided.

APPENDIX D: HARDWARE FAULT REFERENCE

The following diagnostic messages are emitted to the system console when hardware violations occur. They do not terminate the program except where noted.

Fault Message	Trigger Condition	Effect
[HARDWARE FAULT] Segmentation Fault: Read out of bounds	Memory Module receives a read to address ≥ 0x0F00	Returns 0xFF; execution continues
[HARDWARE FAULT] Segmentation Fault: Write out of bounds	Memory Module receives a write to address ≥ 0x0F00	Write discarded; execution continues
[MOTHERBOARD FAULT] Invalid Read Address	Address decoder receives a read to an unmapped address	0x00 placed on Data Pathway; execution continues
[MOTHERBOARD FAULT] Invalid Write Address	Address decoder receives a write to an unmapped address	Write discarded; execution continues

[HARDWARE FAULT] Invalid Instruction	Decode Matrix lookup returns IS_VALID = false	Cycle aborted; execution continues at next PC
[CRITICAL WARNING] Power Draw exceeded PSU Capacity	Polled wattage exceeds PSU rated capacity	PSU calls killPower() on the Mainboard — permanent halt
[CRITICAL ERROR] Thermal threshold exceeded	Processor temperature exceeds 90.0°C	Sets HF bit in FLAGS — permanent halt
[Hardware Interrupt] Awaiting Keyboard Input	MMIO read to 0x0FF0 or 0x0FF3 with empty buffer	Clock suspended until user types; execution resumes after input